ATTRIBUTION-PATCHING-GUIDED REPLAY REFINEMENT FOR TASK-VECTOR MERGING

Anonymous authors
Paper under double-blind review

ABSTRACT

We study model merging when only a small labeled replay buffer is available for each task. We introduce expert-anchored gradient descent, a replay refinement that can start from any merge or directly from the pretrained model. For each task, a first-order loss attribution selects coordinates where moving toward its expert is locally beneficial; updates are scaled by their distance to the expert and clipped to prevent overshoot. This yields equivalent attribution-patching and trust-region interpretations and is provably non-expansive toward the experts' box hull at any learning rate. We evaluate the method on multi-head GLUE-8, eight- and twenty-task CLIP suites, and a decoder-only MergeBench track, using at most sixteen labeled examples per task and tuning only on held-out buffers. From the pretrained model, the anchored update performs best on every benchmark; from merge initializations, it typically improves performance and reduces sensitivity to the initial merge. With eight examples per task, it still outperforms tuned task arithmetic while unconstrained replay descent often overfits. On tasks never merged or refined, it retains more of the pretrained model's zero-shot ability than any checkpoint-only merge or unconstrained replay descent.

1 INTRODUCTION

Merging fine-tuned variants of a shared pretrained model combines their skills in weight space, without retraining, ensembling, or routing. But merges degrade under task interference, and the data-free repairs—sparsification, sign election, learned coefficients—plateau well below the experts. In many practical settings a small *labeled* replay buffer per task is available (it is routinely retained for validation or continual learning); the question is how to spend such a tiny labeled budget so that it repairs the merge without giving up what made merging attractive: a single model, low cost, and bounded drift from checkpoints one already trusts.

We refine the merged model with replay gradients *anchored to the task experts*: each coordinate's step is kept only where moving toward the expert locally reduces that task's replay loss (an attribution-patching sign), weighted by the coordinate's distance to the expert, and clipped so it can never overshoot—applied task-sequentially, from *any* initialization, including the pretrained model itself. The update is provably non-expansive toward the experts' box hull at any learning rate (Proposition 1).

Our contributions are: (i) a replay-refinement method with two equivalent derivations—parameter-space attribution patching, and a trust-region view of the expert-anchored descent—and a divergence-safety guarantee; (ii) a uniform evaluation across four benchmark families (multi-head GLUE-8, the eight- and twenty-task CLIP suites, and decoder-only LLM merging in the MergeBench setting) under a strict protocol in which every hyperparameter is chosen on a small held-out selection buffer and the evaluation split is never used for tuning, with matched data budgets and sanity controls; (iii) evidence that from the pretrained model—the setting with no merge to inherit—the anchored update is the only one of the three replay families that turns a 16-example buffer into multi-task ability, while from merge initializations it usually improves performance and makes the outcome markedly less sensitive to which merge was handed to it; (iv) a characterization of where the anchor earns its keep: as the buffer shrinks to eight examples per task, unconstrained descent silently overfits—a quarter of its configurations lose test accuracy while their replay loss is still falling—whereas the dedicated safety audit finds no corresponding anchored overfitting signature, although selected composition rows can still decline; and (v) a measurement of what the refinement costs the pretrained model on tasks never

merged or refined, under a train/held-out split fixed by the literature rather than chosen by us: the anchored update dominates both unconstrained controls on accuracy and retention simultaneously, and retains more than every checkpoint-only merge.

2 Related Work and Background

Model merging and task vectors. Model merging combines fine-tuned variants of a shared pretrained model into a single model in weight space, avoiding ensembling overhead: model soups average compatible checkpoints (Wortsman et al., 2022), Fisher merging weights parameters by Fisher information (Matena and Raffel, 2022), and task arithmetic represents fine-tuning as a *task vector* $\tau_t = \theta_t - \theta_0$ and merges as $\theta_0 + \sum_t \lambda_t \tau_t$ (Ilharco et al., 2022)—checkpoint-only, but prone to over-counted directions and sign or magnitude conflicts across tasks.

Interference-aware and data-dependent merging. A major line of work addresses interference between task vectors: TIES-Merging trims small-magnitude updates and merges only sign-consistent components (Yadav et al., 2023); DARE randomly drops most delta parameters and rescales the rest (Yu et al., 2023), generalized by DELLA (Deep et al., 2024); Model Breadcrumbs removes negligible and outlier perturbations (Davari and Belilovsky, 2023); Localize-and-Stitch stitches sparse task-relevant regions back into the pretrained model (He et al., 2024). Most closely related to the trust-region reading of our update (Section 3.2), Task Arithmetic in Trust Region (TATR) uses task-loss gradients at the pretrained model to identify gradient-orthogonal parameter dimensions in which task vectors can be merged with limited interference (Sun et al., 2025). Closest in optimization form, DOGE models merging as adaptive projective gradient descent (APGD): it learns a shared modification to the task vectors under a data-free Taylor objective, projects its gradient orthogonally to an SVD-derived shared subspace, and sets layer-wise merge coefficients from task-vector norms (Wei et al., 2025). TATR and DOGE both restrict the merge itself without labeled data; our method instead evaluates labeled task gradients at the evolving model and takes sequential, coordinate-wise expert-directed refinement steps. A complementary line uses small amounts of data or model outputs: APL scores task vectors by a data-dependent contrast (Kong et al., 2024), RegMean solves each linear layer by regression on its inputs (Jin et al., 2023), and AdaMerging learns coefficients by entropy minimization (Yang et al., 2024). Twin-Merging, Task Vector Bases, and FusionBench extend and standardize this space (Lu et al., 2024; Zeng et al., 2025; Tang et al., 2024). The present work does not introduce another pruning rule: it asks whether a small *labeled* replay buffer can repair an initial merge while staying close to the known experts, using the sign of a merge-to-expert attribution only as a local gate for refinement.

Attribution patching. Attribution patching approximates the effect of an intervention by a first-order Taylor expansion, replacing many explicit interventions with gradient-based scores (Syed et al., 2024; Kramar et al., 2024); in parameter space the analogue is a directional derivative of the task loss along a chosen displacement—here, the movement from the current merged model toward a task expert. Extended positioning against the benchmark families of the merging literature is given in Appendix C.

3 Method

3.1 Problem Setup

Let $\mathcal{T} = \{1, \ldots, T\}$ be a set of tasks whose experts are all initialized from the same pretrained encoder $\theta_0 \in \mathbb{R}^d$; fine-tuning on task t gives expert parameters θ_t and task vector $\tau_t = \theta_t - \theta_0$. Writing $r \in \{1, \ldots, d\}$ for a parameter coordinate, the refinement starts from an *arbitrary* initial model $\theta^{(0)} \in \mathbb{R}^d$: any weight-space merge of the experts—task arithmetic

$$\theta^{(0)} = \theta_0 + \sum_{i=1}^{T} \lambda_i \tau_i \tag{1}$$

with fixed coefficients λ_i is the simplest instance, but TIES, RegMean, or AdaMerging merges serve equally (Section 5), as does the pretrained model itself, $\theta^{(0)} = \theta_0$ (the $\lambda_i = 0$ case; Section 5.1).

Nothing in the method depends on how $\theta^{(0)}$ was constructed; only the experts $\{\theta_i\}$ enter the update. Let $\mathcal{D}_t^{\text{probe}}$ be a small labeled replay buffer used for refinement, $\mathcal{D}_t^{\text{eval}}$ the evaluation split used only for reporting, and $\mathcal{L}_i(\theta; \mathcal{D}_i^{\text{probe}})$ the replay-buffer loss of encoder parameters θ under task i's prediction head. In the multi-head settings task-specific heads are not merged and the merged encoder is evaluated with each task's own head; the shared-output settings merge everything. The method uses no base-relative saliency term and no pre-merge pruning mask: its only data-dependent operation is a short replay-buffer refinement of $\theta^{(0)}$.

3.2 Expert-Anchored Gradient Descent

The natural data-dependent baseline from this initialization is ordinary replay-buffer gradient descent, $\theta^{(s+1)} = \theta^{(s)} - \eta \sum_{i=1}^{T} \nabla_\theta \mathcal{L}_i(\theta^{(s)}; \mathcal{D}_i^{\text{probe}})$, which treats the merge like any other starting point and lets the replay gradients move every coordinate freely, with nothing tying the refined model to the known experts. Our method—*expert-anchored gradient descent*—keeps the descent but anchors it to the experts: for each task i and coordinate r we (i) *weight* the negative-gradient step by the current distance $|v_{i,r}|$ to the expert, so coordinates already close to the expert barely move; (ii) *gate* the step, keeping only coordinates where it moves the model *toward* the expert; and (iii) *clip* the result to the current expert distance, so a single update can never overshoot the expert. The gate and the clip give the update the structure of a classical trust-region method—a step is taken only within a region where the local signal is trusted—except that the region is set by the experts rather than by a tuned radius: for each coordinate it is the interval of radius $|v_{i,r}|$ around the current value, reaching exactly to the expert. We refer to it as the *expert-distance trust region*.

Let $\theta^{(s,0)} = \theta^{(s)}$ and let π_s be the task order for sweep s. For within-sweep index $k \in \{1, \dots, T\}$, set $i = \pi_s(k)$ and compute

$$g_i^{(s,k)} = \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}}), \tag{2}$$

$$v_i^{(s,k)} = \theta_i - \theta^{(s,k-1)}. \tag{3}$$

The update moves each coordinate that passes the gate a capped fraction of its remaining distance to the expert, and leaves every other coordinate unchanged:

$$u_{i,r}^{(s,k)} = \begin{cases} \min\big(\eta\,|g_{i,r}^{(s,k)}|,\, 1\big)\, v_{i,r}^{(s,k)} & \text{if } g_{i,r}^{(s,k)}\, v_{i,r}^{(s,k)} < 0, \\ 0 & \text{otherwise;} \end{cases} \tag{4}$$

the model is updated immediately, $\theta^{(s,k)} = \theta^{(s,k-1)} + u_i^{(s,k)}$, with $\theta^{(s+1)} = \theta^{(s,T)}$ after all T tasks in the sweep have been processed. Equation (4) is the weight–gate–clip step of the previous paragraph written in one line: on gated coordinates the sign condition makes $-g_{i,r}^{(s,k)}$ and $v_{i,r}^{(s,k)}$ agree, so the distance-weighted descent step $-\eta\, g_{i,r}^{(s,k)} |v_{i,r}^{(s,k)}|$ equals $\eta\, |g_{i,r}^{(s,k)}|\, v_{i,r}^{(s,k)}$, and clipping it to the expert-distance trust region caps the step fraction at one. The gate condition $g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0$ keeps only the coordinates whose replay step also moves the current model toward expert i, and is exactly the attribution-patching sign that Section 3.3 derives in Equation (5): the two perspectives agree coordinate by coordinate.

Because the step fraction $\min(\eta\,|g_{i,r}^{(s,k)}|, 1)$ never exceeds one, per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η: the learning rate controls how many coordinates reach the trust-region boundary, never the maximum step. Optionally, $u_i^{(s,k)}$ is RMS-normalized tensor-wise. Algorithm 1 summarizes the procedure.

Algorithm 1 Sequential expert-anchored gradient descent (APR)

Require: Task experts $\{\theta_i\}_{i=1}^{T}$; task heads; replay buffers $\{\mathcal{D}_i^{\text{probe}}\}_{i=1}^{T}$; initial model $\theta^{(0)}$ (any weight-space merge of the experts, or the pretrained model θ_0 itself); steps S; learning rate η.

for $s = 0, \dots, S-1$ **do**
  $\theta^{(s,0)} \leftarrow \theta^{(s)}$.
  Choose a task order π_s over $\{1, \dots, T\}$.
  for $k = 1, \dots, T$ **do**
    $i \leftarrow \pi_s(k)$.
    $g_i^{(s,k)} \leftarrow \nabla_\theta \mathcal{L}_i(\theta^{(s,k-1)}; \mathcal{D}_i^{\text{probe}})$ using task i's head.
    $v_i^{(s,k)} \leftarrow \theta_i - \theta^{(s,k-1)}$.
    for each mergeable coordinate r **do**
      $u_{i,r}^{(s,k)} \leftarrow \min\big(\eta\,|g_{i,r}^{(s,k)}|,\ 1\big)\, v_{i,r}^{(s,k)} \cdot \mathbf{1}\{g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0\}$. {gated, capped step toward θ_i; Eq. (4)}
    end for
    $\theta^{(s,k)} \leftarrow \theta^{(s,k-1)} + u_i^{(s,k)}$.
  end for
  $\theta^{(s+1)} \leftarrow \theta^{(s,T)}$.
end for

return $\theta^{(S)}$.

The informal claim that the refinement "cannot diverge" is made precise by the following guarantee, whose proof (as an instance of a general geometric result, Theorem 1) is given in Appendix A.

Proposition 1 (Non-expansiveness toward the expert box)**.** *Let* $B_{\text{exp}} := \prod_{r=1}^{d} [\min_{j \in \mathcal{T}} \theta_{j,r},\ \max_{j \in \mathcal{T}} \theta_{j,r}]$ *be the axis-aligned box hull of the task experts, and let* $d_p(\cdot, B_{\text{exp}})$ *denote* ℓ^p *distance to it. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every* $1 \le p \le \infty$ *and every sweep* s *and within-sweep index* k*,*

$$d_p\Big(\theta^{(s,k)}, B_{\text{exp}}\Big) \le d_p\Big(\theta^{(s,k-1)}, B_{\text{exp}}\Big),$$

and hence $d_p(\theta^{(S)}, B_{\text{exp}}) \le d_p(\theta^{(0)}, B_{\text{exp}})$ *after any number of sweeps, at any learning rate.*

Every gated, clipped step moves each coordinate between its current value and the corresponding expert coordinate, and such movement can never increase the distance to the box; the box is moreover exactly the ℓ^1-geodesic convex hull of the experts (Proposition 2), not an arbitrary safe region. The guarantee holds for any initialization $\theta^{(0)}$, including the pretrained model.

3.3 A Second Perspective: Attribution Patching in Parameter Space

The same per-coordinate update arises from an interpretability-style argument that treats each move toward a task expert as a parameter-space intervention. Let ϑ denote the current encoder, $v_i(\vartheta) = \theta_i - \vartheta$ the merge-to-expert direction, and $g_i(\vartheta)$ the task gradient under task i's head. The first-order attribution-patching estimate of the loss change from moving ϑ toward expert i is $\Delta_i^{\text{AP}}(\vartheta) = \langle g_i(\vartheta), v_i(\vartheta) \rangle$, whose coordinate-wise contribution $g_{i,r} v_{i,r}$ is negative exactly when moving coordinate r toward the expert is locally predicted to reduce task i's loss. The loss-decreasing expert gate is therefore

$$m_{i,r}(\vartheta) = \mathbf{1}\{g_{i,r}(\vartheta)\, v_{i,r}(\vartheta) < 0\}. \tag{5}$$

The masked, distance-scaled step $-g_{i,r}|v_{i,r}|\, m_{i,r}$, scaled by η and capped at the expert distance, recovers exactly the update of Equation (4): whenever the gate is on, $-g_{i,r}$ and $v_{i,r}$ share a sign, so the update points toward the expert, with a step that shrinks as the coordinate approaches it. The gate certifies only a local first-order effect for task i; cross-task interference must be measured experimentally. After this reading, we abbreviate the update as *APR* (attribution-patching-guided replay) throughout the experiments.

4 Experimental Setup

Settings. We evaluate on four benchmark families. *Multi-head* GLUE-8 (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE) (Wang et al., 2018) with RoBERTa-base (Liu et al., 2019) merges

the shared encoder and evaluates with each task's own trained head—an intentionally *oracle-task-id* setting. A *CLIP/ViT-B/32* vision track uses the standard eight-task task-vector suite (Radford et al., 2021; Ilharco et al., 2022) and its twenty-task extension (Wang et al., 2024), probing a different modality and—since more task vectors then compete for the same encoder weights—a substantially higher-interference regime (Section 5.1). A *decoder-only LLM* track adopts the MergeBench setting (He et al., 2025): published full-parameter domain experts share a common pretrained checkpoint, so task vectors and merge-to-expert directions are well defined and no expert training is required on our side. We use Llama-3.2-3B with the mathematics, coding, instruction-following, and multilingual experts, scored by judge-free automatic metrics, and merge the full model including the LM head—so every task decodes through one shared output and no task identity is assumed at evaluation, the assumption the multi-head setting grants for free. Refinement uses $n \leq 16$ labeled replay examples per task ($n = 8$ on MergeBench), class-balanced where possible and disjoint from both the selection buffer and evaluation; full details appear in Appendix C.

Baselines. Baselines are grouped by data regime: *checkpoint-only* (task arithmetic, TIES, DARE-TIES, Breadcrumbs, DOGE/APGD); *data-dependent but label-free* (AdaMerging); *labeled* (ordinary replay GD from the same initialization, and the ungated distance-scaled update—the two ablations that isolate the anchor and the gate); and *sanity controls* (inverted and density-matched random gates). All three labeled families use the same replay examples, the same selection buffer, and the same hyperparameter search budget. Each merge baseline's own hyperparameters (coefficient, density, sparsity, or global APGD scale) are likewise selected on the selection buffer.

Metrics. Tasks are scored by their standard validation metrics: Matthews correlation for CoLA, correlation for STS-B, matched/mismatched accuracy for MNLI, accuracy or accuracy/F1 elsewhere on GLUE, and top-1 accuracy on the CLIP suites; the MergeBench tasks use judge-free automatic metrics (verifiable instruction constraints, exact match on grade-school mathematics, and functional-correctness pass rates on code). These are not commensurable across tasks, so the primary aggregate throughout is normalized retention,

$$\mathrm{NormRet}_t = \frac{\mathrm{Score}_t(\theta_{\mathrm{merge}}) - \mathrm{Score}_t(\theta_0)}{\mathrm{Score}_t(\theta_t) - \mathrm{Score}_t(\theta_0) + \epsilon_{\mathrm{metric}}}, \tag{6}$$

which places the pretrained model at 0 and the task expert at 1 ($\epsilon_{\mathrm{metric}}$ guards small denominators). Mean and worst-task values are always reported together—worst-task matters because a merge can improve the average while catastrophically harming a small task. Two benchmarks depart from this convention because their retention denominators degenerate: on the twenty-task CLIP suite several experts barely improve on the zero-shot backbone, and on MergeBench the multilingual expert finishes *below* the pretrained model (0.494 vs. 0.495), so a near-zero or negative denominator makes per-task retention explode or invert. Both benchmarks therefore report absolute mean and worst-task accuracy, quoting the pretrained floor and expert ceiling for scale (Section 5.1).

Protocol. Refinement runs S sweeps from the given initialization at a constant learning rate, with the trust-region clip applied per-update (Eq. (4)). Each family—ordinary replay gradient descent (GD), the proposed expert-anchored gradient descent (APR), and the ungated distance-scaled update—receives an equal-size search over the learning rate and the sweep count S. Each task contributes a labeled replay buffer used for the refinement sweeps and a *disjoint* selection buffer of the same size; every hyperparameter—including each merge baseline's own coefficient and density—is chosen by minimizing loss on the selection buffer. We use 8 and 16 examples per task in each buffer.

5 Results

5.1 Performance on the Merged Tasks

Task-level scores at the $8{+}8$ budget are given in Appendix D; the full twenty-task campaign, with its diagnostics and budget grids, is in Appendix E.

The underlying task-level evaluation scores for the three benchmarks at the $8{+}8$ budget are reported in Appendix Tables 5–7.

Table 1: **The experiment grid under split selection at a $16{+}16$ budget**, instantiated identically on every encoder benchmark. Each task contributes 16 labeled replay examples for the refinement sweeps and a *disjoint* 16-example selection buffer (32 labels per task in total). Every hyperparameter—merge coefficient, density, refinement learning rate, and the sweep count S—is chosen by minimizing held-out loss on the selection buffer, and the evaluation split is read *only* at the selected cell, so the table reports what a practitioner obtains without ever touching test data. Entries are mean aggregate with worst-task in parentheses: normalized retention on GLUE-8 and CLIP-8, where the pretrained *alone* row is the zero point, and absolute accuracy on MergeBench and twenty-task CLIP, whose retention denominators degenerate (Section 4). AdaMerging is given the matched budget—the same 16 unlabeled inputs per task that APR sees labeled—rather than the standard transductive formulation that adapts on the evaluation split. DOGE/APGD is checkpoint-only: no replay labels construct its candidates, and the disjoint buffer selects only its global scale η; all three selected scales are interior to their benchmark-specific grids. The horizon grid is $S \in \{5, 20, 50\}$ on CLIP-20 and on the merge-initialized GLUE-8 rows, $S \in \{5, 20, 50, 100, 200\}$ on CLIP-8 and the from-pretrained GLUE-8 rows, and, on MergeBench (run from the pretrained model only at this budget, since generation-based scoring is the dominant cost there; its composition tier is reported at $8{+}8$ in Table 2), $S \in \{1, 2, 3, 5\}$ for GD and $S \in \{2, 3, 5, 8, 12\}$ for APR with the task order re-randomized each sweep—the APR grid was extended upward in both η and S after its first selection landed on the grid edge, and its selected cell is interior on both axes; on the encoder tracks every arm receives the same grid as the arms beside it, so no treatment is searched over a wider space than its controls. †: the selected S is the longest its grid offered, so the cell is a lower bound rather than a verified interior optimum. Dashes: not run.

Initialization	Treatment	GLUE-8	CLIP-8	MergeBench	CLIP-20
		normalized retention		absolute accuracy	
Pretrained	alone	0	0	0.289 (.06)	0.550 (.10)
	+GD	0.209 (−.18)	0.046† (−.36)	0.315 (.11)	0.583† (.12)
	+ungated	−0.018 (−.49)	−0.036 (−.59)	—	0.586 (.11)
	+APR	0.315 (−.12)	0.432 (−.16)	0.372 (.24)	0.647 (.22)
Task arithmetic	alone	0.320 (−.18)	0.270 (−.53)	—	0.495 (.15)
	+GD	0.574 (+.06)	0.420† (−.36)	—	0.555 (.18)
	+ungated	0.586 (−.01)	0.404† (−.49)	—	0.602 (.17)
	+APR	**0.608** (+.10)	0.524† (+.05)	—	0.667 (.25)
Breadcrumbs	alone	0.030 (−2.14)	0.431 (+.08)	—	0.603 (.12)
	+GD	0.534 (+.04)	0.513† (+.04)	—	0.638† (.17)
	+ungated	0.563 (+.04)	0.469 (−.09)	—	0.650 (.19)
	+APR	0.567 (+.04)	0.485† (+.00)	—	0.661 (.20)
DARE-TIES	alone	0.301 (−.34)	0.364 (−.32)	—	0.520 (.20)
	+GD	0.521 (−.08)	0.523† (−.07)	—	0.594† (.28)
	+ungated	0.574 (+.09)	0.291† (−1.85)	—	0.579 (.09)
	+APR	0.586 (+.11)	0.527† (−.39)	—	0.664 (.30)
TIES	alone	0.208 (−.22)	0.525 (+.21)	—	0.619 (.19)
	+GD	0.516 (−.13)	0.600† (+.18)	—	0.658† (.26)
	+ungated	0.525 (−.08)	0.561 (−.20)	—	0.672 (.28)
	+APR	0.543 (−.03)	0.608† (+.11)	—	**0.676** (.27)
DOGE/APGD	alone	0.016 (+.00)	0.138 (−1.93)	—	0.516 (.16)
AdaMerging	alone	0.029 (−3.02)	0.533 (+.09)	—	0.597 (.10)
	+GD	0.459† (−.35)	0.601† (+.14)	—	0.637† (.13)
	+ungated	0.538 (−.36)	0.557† (−.04)	—	0.649 (.16)
	+APR	0.564 (−.02)	**0.638**† (+.32)	—	0.665 (.20)

Reading the grid. Table 1 reports the three encoder benchmarks, and the from-pretrained cells of MergeBench, at a labeled budget of 16 refinement and 16 selection examples per task; Table 2 repeats the three encoder benchmarks at half that budget and reports the full MergeBench grid at its $8{+}8$ budget. Two conventions matter for reading the CLIP-20 column. First, it is scored in absolute top-1 accuracy (zero-shot floor 0.550, expert ceiling 0.896), for the degeneracy reason declared in Section 4; on that scale interference is visibly dominant—the best checkpoint-only merge (TIES) recovers only a fifth of the floor-to-ceiling gap, against roughly half of the expert–base gap at eight tasks (0.525)—which Appendix E traces to the geometry of the twenty-expert compromise. Second, its task-arithmetic row carries a protocol artifact: the coefficient grid offered to selection is the standard eight-task one ($\lambda \in \{0.2, \ldots, 0.5\}$), whose optimum at twenty tasks lies *outside* the grid

Table 2: **The canonical grid under split selection at an $8{+}8$ budget.** Each task contributes eight labeled replay examples and a disjoint eight-example selection buffer. Every hyperparameter is selected on the selection buffer before the evaluation split is read; the encoder tracks use $S \in \{5, 20, 50\}$ with a fixed task order, and MergeBench uses $S \in \{1, 2, 3, 5\}$ with the task order re-randomized each sweep, extended to $S \in \{2, 3, 5, 8, 12\}$ (with η up to 8192) for the from-pretrained and from-task-arithmetic APR cells, whose first selections had landed on the grid edge; every selected MergeBench cell is interior on both axes. DOGE/APGD remains checkpoint-only and uses this buffer only to select η. Entries follow Table 1: mean normalized retention with worst-task retention in parentheses, except MergeBench and CLIP-20, which report absolute accuracy. The decoder-only MergeBench track uses Llama-3.2-3B with mathematics, coding, instruction-following, and multilingual experts; its pretrained floor is 0.289 and expert ceiling 0.548. † marks a selected horizon at the longest value offered; dashes are configurations not run.

Initialization	Treatment	GLUE-8	CLIP-8	MergeBench	CLIP-20
		normalized retention		absolute accuracy	
Pretrained	alone	0	0	0.289 (.06)	0.550 (.10)
	+GD	−0.361 (−3.52)	−0.148 (−1.04)	—	0.574† (.10)
	+ungated	0.089† (−.31)	−0.006 (−.41)	—	0.568 (.08)
	+APR	0.223† (−.33)	0.295† (−.45)	0.373 (.28)	0.645 (.22)
Task arithmetic	alone	0.320 (−.18)	0.270 (−.53)	0.470 (.33)	0.495 (.15)
	+GD	0.530 (+.04)	0.167† (−1.10)	—	0.545 (.18)
	+ungated	0.529 (+.03)	0.241† (−.79)	—	0.576 (.23)
	+APR	0.498 (−.18)	0.462† (−.12)	0.470 (.35)	0.648 (.22)
Breadcrumbs	alone	0.030 (−2.14)	0.431 (+.08)	0.417 (.25)	0.602 (.12)
	+GD	0.512 (−.00)	0.479† (+.04)	—	0.632† (.15)
	+ungated	0.536 (+.05)	0.391† (−.40)	—	0.624 (.17)
	+APR	0.528 (−.06)	0.339† (−.46)	0.412 (.30)	0.652 (.22)
DARE-TIES	alone	0.301 (−.34)	0.364 (−.32)	0.490 (.35)	0.520 (.20)
	+GD	0.484 (−.08)	−1.273† (−4.87)	—	0.571† (.26)
	+ungated	0.534 (+.08)	−0.578 (−3.11)	—	0.499 (.05)
	+APR	**0.576** (+.19)	0.498† (−.31)	0.495 (.37)	0.649† (.27)
TIES	alone	0.208 (−.22)	0.525 (+.21)	0.464 (.36)	0.619 (.19)
	+GD	0.442 (−.10)	0.585† (+.18)	—	0.651† (.25)
	+ungated	0.442 (−.07)	0.318† (−1.57)	—	0.622 (.10)
	+APR	0.489† (+.01)	0.566† (−.08)	0.468 (**.38**)	0.665 (.28)
DOGE/APGD	alone	0.016 (+.00)	0.138 (−1.93)	—	0.516 (.16)
AdaMerging	alone	0.028 (−3.01)	0.463 (−.04)	**0.496** (.36)	0.592 (.10)
	+GD	0.016 (−2.95)	0.502† (−.06)	—	0.623† (.11)
	+ungated	0.466 (−.10)	0.285† (−1.41)	—	0.563† (.18)
	+APR	0.543 (+.02)	0.554† (+.15)	0.495 (.37)	0.640 (.19)
RegMean	alone	0.477 (+.06)	0.627 (+.32)	—	0.697 (.21)
	+GD	0.480 (+.06)	0.611† (+.29)	—	0.707† (.23)
	+ungated	0.462 (+.07)	0.622† (+.28)	—	**0.709** (.24)
	+APR	0.517 (+.10)	**0.632**† (+.31)	—	0.709 (.24)

($\lambda \approx 0.05$–0.08, accuracy 0.604; Appendix E), so the selected $\lambda = 0.2$ is a boundary pick and that merge lands below the zero-shot floor (0.495).

Cold start: refining from the pretrained model. Refining *from the pretrained model*—the setting with no merge to inherit, and the one where the refinement must supply all of the multi-task ability itself—is an unambiguous win for the gated, expert-anchored update: APR is the best cold-start treatment on all four benchmarks. On the three encoder tracks it is ahead of the better of the two unconstrained baselines by 0.06 (CLIP-20), 0.11 (GLUE-8), and 0.39 (CLIP-8), and neither unconstrained baseline is viable at this budget: ordinary descent recovers between a twentieth and a fifth of the expert–base gap on the two retention benchmarks (0.046 on CLIP-8, 0.209 on GLUE-8), and the ungated update fails to improve on the pretrained model at all (-0.018 on GLUE-8, -0.036 on CLIP-8), so distance scaling without the gate is not merely weaker than APR but useless from this initialization. On the decoder track, both labeled arms improve the pretrained Llama-3.2-3B at sixteen labels per task—mean accuracy $0.289 \to 0.315$ (GD) and $\to 0.372$ (APR) against an expert ceiling of 0.548 (Table 1)—and APR's margin is widest where the pretrained model is weakest: mathematics rises from 0.063 to 0.317 under APR against 0.107 under GD, and APR's worst task (0.240) is more

than twice GD's (0.107). At eight labels per task (Table 2) APR reaches 0.373 with a worst task of 0.283, so the decoder result does not hinge on the larger budget. Only the gated, distance-scaled update turns a small replay buffer into multi-task ability with no merge to build on.

Composition: refining from a merge. From merge initializations the refinement is a complement to merging rather than a replacement. At the 16+16 budget it improves every initialization on all three encoder benchmarks, and APR is the strongest of the three arms in every row on GLUE-8 and CLIP-20; on CLIP-8 it leads four of the five merge rows (ordinary descent edges it from Breadcrumbs, 0.513 vs. 0.485). Refinement also makes the outcome far less sensitive to which merge it was handed: on CLIP-20 the unrefined merges span 0.495–0.619 while the +APR rows span 0.661–0.676, and on GLUE-8 a spread of 0.029–0.320 closes to 0.543–0.608. At the halved budget of Table 2 the ordering is less uniform—an unconstrained arm edges APR in a few rows (TIES and Breadcrumbs on CLIP-8, task arithmetic and Breadcrumbs on GLUE-8)—but the unconstrained arms now carry a tail risk the anchored update never exhibits: from DARE-TIES on CLIP-8, descent collapses to -1.27 mean retention (-4.87 worst-task) and the ungated update to -0.58, while APR improves the same initialization to 0.498. On the decoder track, composition is substantially stronger than cold start (0.41–0.50 against 0.37), and what refinement adds to a merge there is worst-task repair rather than mean accuracy: from every launch point the weakest task (instruction following) rises—task arithmetic $0.333 \to 0.350$, DARE-TIES $0.350 \to 0.367$, TIES $0.357 \to 0.377$, AdaMerging $0.357 \to 0.373$, Breadcrumbs $0.253 \to 0.303$—while the mean moves by at most 0.005 (DARE-TIES, $0.490 \to 0.495$) and is flat or nominally lower from task arithmetic ($0.470 \to 0.470$), AdaMerging ($0.496 \to 0.495$), and Breadcrumbs ($0.417 \to 0.412$). All five decoder composition cells select short horizons ($S \leq 2$, except $S = 5$ from task arithmetic); the mean changes are within the resolution of 300-item generation evaluations, and the worst-task repair is the consistent signal.

Further baselines: DOGE/APGD, RegMean, and transductive AdaMerging. The added DOGE/APGD row is the closest checkpoint-only optimization comparison and uses neither replay buffer to construct its checkpoint: n affects only held-out selection of η. Both budgets select the same interior scale on every benchmark, so their evaluated checkpoints and scores coincide. Under this strict selection rule DOGE/APGD is not competitive here: mean retention is 0.016 on GLUE-8 and 0.138 on CLIP-8 (worst-task retention -1.93), while mean accuracy is 0.516 on CLIP-20. We report the selected cells rather than substituting the published fixed scale after seeing evaluation results: on CLIP-8, for example, held-out loss selects $\eta = 0.09$ even though the resulting aggregate is weak. This makes visible a practical mismatch between tiny-buffer cross-entropy selection and the final multi-task aggregate rather than tuning the baseline on test accuracy.

RegMean supplies a substantially stronger data-fitted starting point at the 8+8 budget: its mean retention is 0.477 on GLUE-8 and 0.627 on CLIP-8, and its mean accuracy is 0.697 on CLIP-20. Refinement improves it on all three suites, but by less than it improves checkpoint-space merges: APR reaches 0.517 and 0.632 retention on GLUE-8 and CLIP-8, while on CLIP-20 APR and the ungated anchored update tie at 0.709 mean accuracy. The ordinary-GD cell is effectively unchanged on the retention benchmarks and trails both anchored arms on CLIP-20. The selected RegMean refinement rates (0.5, 1, and 0.125 for APR) are smaller than those used from checkpoint-space initializations, consistent with the far-initialization behavior examined in Appendix E.

For completeness, we also ran the conventional *transductive* AdaMerging protocol, in which its entropy objective sees the evaluation inputs. These cells are an audit rather than part of the strict split-selected comparison above. On CLIP-8, transductive AdaMerging-layer scores 0.604 mean retention and APR raises it to 0.645 at both budgets ($n = 8$: worst 0.288; $n = 16$: worst 0.206). On GLUE-8 the transductive merge itself is weak (0.025 mean retention for task-wise coefficients), but APR recovers to 0.537 at $n = 8$ and 0.546 at $n = 16$. Thus allowing evaluation inputs strengthens the CLIP initialization but does not change the composition conclusion; because it violates the paper's no-evaluation-data protocol, none of these values replaces the matched-budget AdaMerging rows in Tables 1–2.

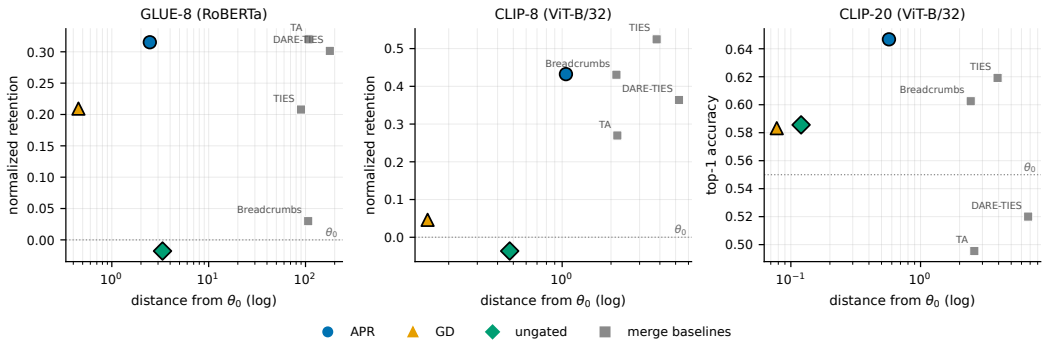

Figure 1: Where good solutions sit relative to the pretrained model, on a single absolute frame: x is the distance from θ_0 (log), y the absolute score, so every point shares one origin. Coloured markers are the three split-selected *from-pretrained* cells of Table 1; gray squares are the merge baselines at their own θ_0-distance. Only from-pretrained cells are plotted, because a refinement launched from a merge inherits that merge's score and is displaced from a different origin, so such points share no common frame with these and their comparison is made in Table 1 instead. Refinement is a small-displacement operation relative to merging: on GLUE-8 the tuned merge sits ≈ 108 from θ_0 for retention 0.320, while APR reaches 0.315 at distance 2.5—matching it at $\approx 2\%$ of the drift—and on all three benchmarks APR lies up and to the right of both unconstrained arms, converting distance into score where they do not. The gray squares are label-free, and the comparison they support is drift at comparable score rather than a claim that from-pretrained refinement outscores every merge: on CLIP-8 the best of them (TIES, 0.525) is above APR's 0.432, at $\approx 3.6\times$ the distance, and refinement launched *from* that merge is the stronger comparison, reported in Table 1. Parameter distance is comparable in magnitude but is not a drift proxy *across* merge families with different geometry (RegMean sits hundreds of units out while retaining well; Appendix E), so whether these small displacements preserve pretrained capability is measured rather than inferred, in Table 3 and the held-out-task study below.

5.2 Retention of Pretrained Capabilities

Strong performance on the merged tasks is useful only if it does not erase capabilities inherited from the pretrained model. We test this first on the pretraining objective itself and then on tasks that are never merged or used for refinement.

Drift on the pretraining objective. Figure 1 shows the refinement reaching merge-level scores at a small fraction of the merge's displacement, but parameter distance is a geometric quantity, not a capability measure. We therefore measure drift on the pretraining objective itself: masked-language-model loss on WikiText-2 validation text. Each refined encoder is evaluated under RoBERTa's original pretrained MLM head; the head is never modified by any experiment, and the WikiText-2 text is never used for refinement or selection, so any increase in this loss over the pretrained model is attributable to encoder drift alone. The probed states are exactly the split-selected from-pretrained cells of Table 1, re-derived from the same replay buffer, with the pretrained model and the task-arithmetic merge as references (Table 3).

With the split-selected merges of Table 1 probed alongside (Table 3), the picture parallels the held-out-task study below. APR is the only point that combines merge-level task retention with sub-merge drift: at 0.315 it matches the best-scoring merges (task arithmetic 0.320, DARE-TIES 0.301) while adding $+1.08$ to the pretraining loss against their $+2.51$ and $+1.59$—the same multitask ability for 43% of task arithmetic's damage, at roughly 2% of its parameter displacement—and it dominates Breadcrumbs and AdaMerging outright (equal or lower drift at ten times the retention). The exception is TIES, which drifts least of every data-free merge ($+0.37$) but at a third less task retention (0.208); the frontier therefore consists of θ_0, TIES, APR, and—by a 0.005 retention margin at more than twice the drift—task arithmetic. Refinement from the pretrained model is thus not the minimum-drift point, but it is the only one at the high-retention end that costs the base model less than half of what a merge at that level does. The two ablation arms add a caution and an attribution. The caution is that

Table 3: Pretraining-capability drift on multi-head GLUE-8 at the 16+16 budget. Merge rows are the split-selected merges of Table 1 (same hyperparameters; the same matched-budget AdaMerging state); refinement rows are its split-selected from-pretrained cells, re-derived from the same replay buffer. "MLM loss" is masked-LM loss on WikiText-2 validation text, computed with RoBERTa's original pretrained MLM head over each row's encoder; the head is never modified and the text is never used for refinement or selection, so the increase over θ_0 is encoder drift alone, measured in the base model's own objective. Single seed. Lower MLM loss and higher retention are both better; * marks the retention/drift Pareto frontier. GD and the ungated update are ablations of APR, listed for attribution.

State	Dist. θ_0	GLUE-8 NormRet	MLM loss (Δ)
Pretrained $\theta_0{}^*$	0.00	0	2.51 (0.00)
Task arithmetic (λ=0.3)*	107.7	**0.320**	5.02 (+2.51)
TIES*	89.8	0.208	**2.88** (+0.37)
DARE-TIES	178.3	0.301	4.10 (+1.59)
Breadcrumbs	106.6	0.030	3.60 (+1.09)
AdaMerging (matched)	116.0	0.029	8.72 (+6.21)
APR from θ_0 (η=8, S=50)*	2.47	0.315	3.59 (+1.08)
GD ablation (η=10^{-3}, S=50)	0.45	0.209	4.13 (+1.62)
ungated ablation (η=8, S=50)	5.32	−0.431	16.10 (+13.59)

displacement does not predict this drift even between refinements: APR ends $5.5\times$ farther from θ_0 than ordinary descent yet drifts *less* in function ($+1.08$ vs. $+1.62$), which is why the drift is measured on the objective rather than proxied by distance. The attribution is that the gate, not the anchor alone, does the protecting: the ungated update, identical except for the gate and run at the same rate, inflates the pretraining loss to 16.10 against the base model's 2.51. The probe is a single seed on a single corpus for one benchmark, and it measures the pretraining objective rather than downstream ability; retained *ability* is measured next.

Held-out-task retention. The drift probe measures retention in the base model's own objective; what matters downstream is retained *ability*: whether the refined model still performs tasks that were never merged or refined. For this we move to the vision track and split the twenty-task suite along a boundary fixed by the literature rather than by us: train on exactly the eight-task CLIP suite, hold out the twelve remaining tasks, evaluated purely zero-shot through frozen text-derived heads so that nothing task-specific leaks. The budget is $n = 8$ and every refinement cell is pinned at the split-selected (η, S) of the corresponding CLIP-8 experiment, so nothing is tuned or selected inside the study. Seven held-out tasks have meaningful base accuracy (mean 0.780) and form the primary aggregate; the five at or near chance—KMNIST and EMNIST at ≈ 0.10, the binary PCAM and RenderedSST2 barely above the coin flip, FER2013 marginal—are recorded but reported separately, by a rule fixed before the run: there is nothing to retain on a chance-level task, and averaging them in dilutes every drop by roughly a third. Only refinement from the pretrained model is reported, since a merge initialization carries no retention guarantee of its own and a composed pipeline's held-out score would confound the two.

Table 4 splits in two. Against the checkpoint-only merges, APR retains more than every one of them—a -0.047 held-out drop against -0.064 (Breadcrumbs), -0.077 (TIES), -0.151 (task arithmetic), and -0.166 (DARE-TIES)—and in the task-arithmetic case it dominates outright, with more training accuracy *and* more retention. Against the strongest label-free merge it does not: at eight training tasks the interference merging has to repair is mild, AdaMerging reaches 0.720 training accuracy at a -0.029 drop, and it dominates from-pretrained refinement on both axes, so the frontier consists of the base model, AdaMerging, and TIES, and contains no refinement point. This holds at matched data: AdaMerging is fitted here from the same eight unlabeled inputs per task that APR sees labeled. Its standard *transductive* formulation—entropy minimised on the full evaluation sets of the eight training tasks, $87{,}433$ unlabeled images against APR's 64 labeled examples—reaches 0.759 at a -0.031 drop instead, so the extra data buys ≈ 0.04 of training accuracy on the split it adapts to and essentially nothing in retention: the retention ordering is a property of the objective rather than of data access. The honest reading is that the refinement buys retention relative to checkpoint-only merging (and,

Table 4: Held-out retention under the literature-fixed split: train on the eight-task CLIP suite, hold out the twelve remaining tasks of the twenty-task suite, evaluated zero-shot through frozen heads. $n = 8$, single seed; every refinement cell is pinned at the split-selected (η, S) of the corresponding CLIP-8 8+8 experiment (constant rate, fixed order), and the merge hyperparameters are those experiments' selections, so nothing is tuned inside this study. "Good-7" averages the seven held-out tasks with meaningful base accuracy (rule fixed before the run); the five near-chance tasks are excluded from the primary aggregate and dilute every drop by roughly a third when averaged in (all-12 base 0.599). Only refinement from the pretrained model is listed: an initialization that is itself a merge carries no retention guarantee, so a composed pipeline's held-out score confounds what the refinement did with what it inherited. Every data-dependent method here draws on the *same* eight unlabeled inputs per task that APR sees labeled—AdaMerging's entropy objective and RegMean's Gram matrices included—so no row is given data the others are denied. * marks the train/good-7 Pareto frontier.

Model (built from the 8 train tasks)	Dist. θ_0	Train-8	Good-7 held-out (drop)
Base model θ_0 *	0.00	0.478	0.780 (0.000)
GD from base (η=10^{-4}, S=20)	0.13	0.496	0.697 (−0.082)
Ungated from base (η=1, S=20)	0.49	0.500	0.699 (−0.080)
APR from base (η=16, S=50)	0.85	0.678	0.732 (−0.047)
AdaMerging-layer*	1.48	0.720	**0.751** (−0.029)
Task arithmetic (λ=0.3)	2.19	0.673	0.628 (−0.151)
Breadcrumbs	2.16	0.705	0.716 (−0.064)
TIES*	3.82	**0.741**	0.702 (−0.077)
DARE-TIES	5.25	0.708	0.613 (−0.166)
RegMean	123	0.572	0.461 (−0.318)

below, relative to unconstrained use of the same labels), not that it is the best available operating point wherever interference is mild. Within the labeled tier, the two ablation arms confirm that this retention belongs to the anchored update rather than to small labeled budgets per se: at the same pinned budget, ordinary descent and the ungated update reach ≈ 0.18 less training accuracy than APR while losing *more* held-out accuracy (-0.082 and -0.080 against -0.047). RegMean reverses relative to its behaviour at larger Gram budgets, finishing worst-retaining in the table (-0.318 at eight inputs), so its inference-invisible displacement is budget-dependent rather than a general property.

Summary. Five claims survive the campaign at $n \leq 16$: (i) the refinement is broadly compositional, improving most initializations—on MergeBench chiefly in the worst task—while the small-buffer Breadcrumbs rows delimit any universal improvement claim; (ii) the anchored, distance-scaled, clipped step is the load-bearing ingredient, the gate contributing worst-task repair, small-budget stability, and held-out retention rather than mean accuracy; (iii) in the dedicated safety audit, anchored runs avoid the overfitting signature that appears in a quarter of unconstrained descent's small-buffer configurations, although selected composition rows can still decline; (iv) eight labels per task suffice to beat the tuned task-arithmetic merge starting from the pretrained model alone; and (v) on held-out tasks never merged or refined, the anchored update dominates both unconstrained controls on accuracy and retention at once and retains more than every checkpoint-only merge, though where interference is mild the strongest label-free merge retains more still (Table 4).

6 Discussion

This work develops an attribution-patching perspective on replay-guided merging: the sign of $g_{i,r}v_{i,r}$ tells, per coordinate, whether moving toward expert i locally reduces replay loss; the same condition is a trust-region acceptance rule, and both views motivate one procedure—mask, distance-scale, clip, apply task-sequentially. Evaluated under a protocol that never reads the evaluation split at 16 labeled examples per task or fewer, the refinement is a *complement* to merging: it typically improves its initialization, reduces sensitivity to which merge it was handed, and can repair the worst task even when the mean is flat. Its decisive setting is the one with no merge to inherit—from the pretrained model the gated, expert-anchored update is the best treatment on every benchmark reported, while neither unconstrained baseline is viable at this budget—and its advantage grows as the buffer shrinks, where descent overfits while its replay loss still improves and the dedicated safety audit finds no corresponding anchored overfitting signature. Refining from the pretrained model also turns out to be cheap in what it costs the pretrained model: it reaches merge-level multitask retention while inflating the base objective's loss by well under half of what the merges at that level do (only TIES drifts

less, at markedly lower task retention), and it does so while moving *farther* in parameter space than ordinary descent, which drifts more—so the anchor buys a better-directed path rather than a shorter one. The same conclusion holds one level up, on retained *ability* rather than the pretraining objective: on tasks never merged or refined, the anchored update dominates both unconstrained controls on accuracy and retention simultaneously and retains more than every checkpoint-only merge, with the gate—not just the anchor—supplying the retention. It does not dominate everything: where interference is mild enough that merging has little to repair, the strongest label-free merge retains more than refinement from θ_0 does. The result is a lightweight bridge between checkpoint-only task arithmetic and full multitask replay fine-tuning, usable at a labeled budget of a few dozen examples per task.

AI USE STATEMENT

In this work, we used generative AI tools to assist with writing and refactoring experiment code and with editing and polishing the text. We did not use generative AI tools for research ideation, experimental design, or data analysis. We have reviewed all AI-assisted work and take responsibility for the final content of this work, including any text, claims, or artifacts produced with the aid of generative AI.

REFERENCES

Mohammad Davari and Eugene Belilovsky. Model Breadcrumbs: Scaling multi-task model merging with sparse masks. *arXiv preprint arXiv:2312.06795*, 2023.

Pala Tej Deep, Rishabh Bhardwaj, and Soujanya Poria. DELLA-Merging: Reducing interference in model merging through magnitude-based sampling. *arXiv preprint arXiv:2406.11617*, 2024.

Shwai He, Runqi Wang, Jindong Wang, Jindong Gu, and Xing Xie. Localize-and-Stitch: Efficient model merging via sparse task arithmetic. *arXiv preprint arXiv:2408.13656*, 2024.

Yifei He, Siqi Zeng, Yuzheng Hu, Rui Yang, Tong Zhang, and Han Zhao. MergeBench: A benchmark for merging domain-specialized LLMs. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2025.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *International Conference on Learning Representations*, 2023.

Fanshuang Kong, Richong Zhang, and Ziqiao Wang. Activated parameter locating via causal intervention for model merging. *arXiv preprint arXiv:2408.09485*, 2024.

Janos Kramar, Tom Lieberum, Rohin Shah, and Neel Nanda. AtP*: An efficient and scalable method for localizing language model behavior to components. *arXiv preprint arXiv:2403.00745*, 2024.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Zhenyi Lu, Chengxu Yin, Wujie Wang, and Xuebo Liu. Twin-Merging: Dynamic integration of modular expertise in model merging. *arXiv preprint arXiv:2406.15479*, 2024.

Michael S. Matena and Colin A. Raffel. Merging models with Fisher-weighted averaging. In *Advances in Neural Information Processing Systems*, 2022.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.

Wenju Sun, Qingyong Li, Wen Wang, Yangli-ao Geng, and Boyang Li. Task Arithmetic in Trust Region: A training-free model merging approach to navigate knowledge conflicts. *arXiv preprint arXiv:2501.15065*, 2025.

Aaquib Syed, Can Rager, and Arthur Conmy. Attribution patching outperforms automated circuit discovery. *arXiv preprint arXiv:2310.10348*, 2024.

Anke Tang, Li Shen, Yong Luo, Liang Ding, Han Hu, Bo Du, and Dacheng Tao. FusionBench: A comprehensive benchmark of deep model fusion. *arXiv preprint arXiv:2406.03280*, 2024.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP*, 2018.

Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Advances in Neural Information Processing Systems*, 2019.

Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jiménez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. In *International Conference on Machine Learning*, 2024.

Yongxian Wei, Anke Tang, Li Shen, Zixuan Hu, Chun Yuan, and Xiaochun Cao. Modeling multi-task model merging as adaptive projective gradient descent. In *International Conference on Machine Learning*, 2025.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, 2022.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *arXiv preprint arXiv:2306.01708*, 2023.

Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. AdaMerging: Adaptive model merging for multi-task learning. *arXiv preprint arXiv:2310.02575*, 2024.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch. *arXiv preprint arXiv:2311.03099*, 2023.

Zihan Zeng, Jiahui Chen, Lei Li, and Min Zhang. Efficient model editing with task vector bases: A theoretical framework and scalable approach. *arXiv preprint arXiv:2502.01015*, 2025.

A Box-Distance Monotonicity and the Expert Box

This appendix proves the general geometric result behind Corollary 1. The statement is self-contained: $x_1, \ldots, x_n$ are arbitrary points in $\mathbb{R}^D$ (in the application, the task experts $\theta_1, \ldots, \theta_T$), and the superscript d indexes coordinates.

Theorem 1 (Box-distance monotonicity under coordinatewise movement toward a hull point)**.** *Let $x_1, \ldots, x_n \in \mathbb{R}^D$. Define the axis-aligned box hull*

$$B := \prod_{d=1}^{D} [m_d, M_d], \qquad m_d := \min_{1 \le j \le n} x_j^d, \qquad M_d := \max_{1 \le j \le n} x_j^d.$$

Let $y, z \in \mathbb{R}^D$. Suppose that there exists some $i \in \{1, \ldots, n\}$ such that, for every coordinate $d \in \{1, \ldots, D\}$, the point z^d lies between x_i^d and y^d. Equivalently,

$$|z^d - x_i^d| \le |y^d - x_i^d|$$

and $z^d - x_i^d$ has the same sign as $y^d - x_i^d$, allowing equality.

Then, for every $1 \le p \le \infty$,

$$d_p(z, B) \le d_p(y, B),$$

where $d_p(u, B)$ denotes the distance from u to B with respect to the ℓ^p-norm.

Proof. For each coordinate d, set

$$I_d := [m_d, M_d].$$

Since $x_i \in B$, we have

$$x_i^d \in I_d$$

for every d.

Define the one-dimensional distance function

$$\delta_d(t) := \operatorname{dist}(t, I_d) = \inf_{s \in I_d} |t - s|.$$

We first prove the following elementary one-dimensional claim.

Claim. Let $I = [m, M] \subset \mathbb{R}$, let $c \in I$, and suppose that z lies between c and y. Then

$$\operatorname{dist}(z, I) \le \operatorname{dist}(y, I).$$

Indeed, if $y \in I$, then since $c \in I$ and I is an interval, every point between c and y also lies in I. Hence $z \in I$, so

$$\operatorname{dist}(z, I) = 0 = \operatorname{dist}(y, I).$$

If $y > M$, then

$$\operatorname{dist}(y, I) = y - M.$$

Since z lies between $c \in I$ and $y > M$, we have $z \le y$. If $z \le M$, then $\operatorname{dist}(z, I) = 0$. If $z > M$, then

$$\operatorname{dist}(z, I) = z - M \le y - M = \operatorname{dist}(y, I).$$

The case $y < m$ is symmetric. This proves the claim.

Now apply the claim coordinatewise with

$$I = I_d, \qquad c = x_i^d.$$

Because z^d lies between x_i^d and y^d, we obtain

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every $d \in \{1, \dots, D\}$.

Next, distance to an axis-aligned box separates coordinatewise. Namely, for $1 \le p < \infty$,

$$d_p(u, B) = \left(\sum_{d=1}^{D} \delta_d(u^d)^p \right)^{1/p},$$

and for $p = \infty$,

$$d_\infty(u, B) = \max_{1 \le d \le D} \delta_d(u^d).$$

This follows because the closest point of B to u is obtained by clipping each coordinate u^d to the interval $I_d = [m_d, M_d]$.

Since

$$\delta_d(z^d) \le \delta_d(y^d)$$

for every d, monotonicity of the ℓ^p-norm on the nonnegative orthant gives

$$d_p(z, B) \le d_p(y, B).$$

This proves the theorem. □

The box hull B may look like a loose relaxation of the ordinary convex hull of the experts, but it is canonical in its own right: it is the smallest set containing the experts that is convex with respect to ℓ^1-geodesics.

Proposition 2. *The set B is the ℓ^1-geodesic convex hull of $\{x_1, \dots, x_n\}$.*

Proof. For $a, b \in \mathbb{R}^D$, define the ℓ^1-metric interval between a and b by

$$I_1(a, b) := \{u \in \mathbb{R}^D : \|a - u\|_1 + \|u - b\|_1 = \|a - b\|_1\}.$$

We claim that

$$I_1(a, b) = \prod_{d=1}^{D} [\min(a^d, b^d), \max(a^d, b^d)].$$

Indeed, by the triangle inequality in one dimension,

$$|a^d - u^d| + |u^d - b^d| \geq |a^d - b^d|$$

for each coordinate d. Summing over d, equality in the ℓ^1-triangle inequality holds if and only if equality holds in every coordinate. In one dimension, equality holds precisely when u^d lies between a^d and b^d. Hence the displayed formula for $I_1(a, b)$ follows.

Now B is clearly ℓ^1-geodesically convex: if $a, b \in B$, then every point coordinatewise between a and b also lies in B.

It remains to show minimality. Let $C \subseteq \mathbb{R}^D$ be any ℓ^1-geodesically convex set containing $x_1, \dots, x_n$. Since C is closed under ℓ^1-metric intervals, it is closed under taking coordinatewise boxes between pairs of its points. In particular, if $a, b \in C$, then both the coordinatewise minimum

$$a \wedge b := (\min(a^1, b^1), \dots, \min(a^D, b^D))$$

and the coordinatewise maximum

$$a \vee b := (\max(a^1, b^1), \dots, \max(a^D, b^D))$$

belong to C.

By induction, C therefore contains the coordinatewise minimum

$$m := (m_1, \dots, m_D)$$

and the coordinatewise maximum

$$M := (M_1, \dots, M_D)$$

of the points $x_1, \dots, x_n$. Since C is ℓ^1-geodesically convex, it contains the entire ℓ^1-metric interval between m and M. But

$$I_1(m, M) = \prod_{d=1}^{D} [m_d, M_d] = B.$$

Therefore every ℓ^1-geodesically convex set containing $x_1, \dots, x_n$ also contains B. Hence B is the smallest such set, i.e.

$$B = \mathrm{gconv}_{\ell^1}\{x_1, \dots, x_n\}.$$

□

We can now state and prove the guarantee for Algorithm 1 used in Section 3.2.

Corollary 1 (Box-distance monotonicity of the sequential refinement)**.** *Let*

$$B_{\exp} := \prod_{r=1}^{d} [\underline{\theta}_r, \overline{\theta}_r], \qquad \underline{\theta}_r := \min_{j \in \mathcal{T}} \theta_{j,r}, \qquad \overline{\theta}_r := \max_{j \in \mathcal{T}} \theta_{j,r},$$

be the axis-aligned box hull of the task experts. Then every within-sweep update of Algorithm 1 is non-expansive in distance to the expert box: for every $1 \leq p \leq \infty$,

$$d_p\left(\theta^{(s,k)}, B_{\exp}\right) \leq d_p\left(\theta^{(s,k-1)}, B_{\exp}\right).$$

Consequently, after any number S of sweeps,

$$d_p\left(\theta^{(S)}, B_{\exp}\right) \leq d_p\left(\theta^{(0)}, B_{\exp}\right) \qquad \textit{for every } 1 \leq p \leq \infty.$$

Proof. Fix a sweep s, a within-sweep index k, and let $i = \pi_s(k)$. We first verify the hypothesis of Theorem 1 with

$$y = \theta^{(s,k-1)}, \qquad z = \theta^{(s,k)}, \qquad x_i = \theta_i.$$

It is enough to check that, for every coordinate r, the new coordinate $\theta_r^{(s,k)}$ lies between the previous coordinate $\theta_r^{(s,k-1)}$ and the expert coordinate $\theta_{i,r}$.

By Equation (4), every coordinate update has the form

$$u_{i,r}^{(s,k)} = \alpha_{i,r}^{(s,k)} v_{i,r}^{(s,k)}, \qquad \alpha_{i,r}^{(s,k)} = \begin{cases} \min\big(\eta\,|g_{i,r}^{(s,k)}|,\, 1\big) & \text{if } g_{i,r}^{(s,k)} v_{i,r}^{(s,k)} < 0, \\ 0 & \text{otherwise,} \end{cases}$$

with $\alpha_{i,r}^{(s,k)} \in [0,1]$ in both cases since $\eta \geq 0$. The same representation is assumed directly in the optional post-processing case. Hence

$$\theta_r^{(s,k)} = \theta_r^{(s,k-1)} + \alpha_{i,r}^{(s,k)} \big(\theta_{i,r} - \theta_r^{(s,k-1)}\big),$$

so $\theta_r^{(s,k)}$ lies between $\theta_r^{(s,k-1)}$ and $\theta_{i,r}$.

The box $B_{\exp}$ is exactly the axis-aligned hull of the expert points $\theta_1, \ldots, \theta_T$. Therefore Theorem 1 gives

$$d_p\Big(\theta^{(s,k)}, B_{\exp}\Big) \leq d_p\Big(\theta^{(s,k-1)}, B_{\exp}\Big)$$

for every $1 \leq p \leq \infty$. Chaining this inequality over $k = 1, \ldots, T$ and over sweeps $s = 0, \ldots, S-1$ gives the final statement. □

B Method Details: Sequential Scheduling

This appendix details the scheduling of the per-task updates, summarized in Section 3.2.

A secondary design choice concerns how the per-task updates are scheduled within a refinement sweep. An aggregated expert-guided update would compute all u_i at $\theta^{(s)}$ and then apply $\eta \sum_i u_i$ at once; we instead apply each task's update immediately after computing it. This has two motivations. First, each update is applied at the same local model state on which its gradient, expert direction, and gate were computed. Second, the trust-region clipping in Equation (4) controls each single-task movement before the next task sees the modified model, rather than allowing several task vectors to accumulate into one larger coordinate step.

Because the clip is applied *after* the learning-rate scaling, the per-coordinate movement is bounded by $|v_{i,r}^{(s,k)}|$ for any η; a single update can never overshoot the expert in a coordinate, so the refinement stays within the expert-distance trust region no matter how large η is, and the learning rate controls only how many coordinates are driven to the trust-region boundary. The sequential rule still does not guarantee Pareto improvement across tasks, because an update that helps task i can hurt task j; we therefore treat task ordering, cross-task interference, and worst-task retention as explicit diagnostics rather than as consequences of the gate alone. In practice, one can also normalize $u_i^{(s,k)}$ by tensor-wise RMS before applying the single-task update.

C Extended Experimental Protocol

This appendix expands the protocol of Section 4: benchmark positioning, secondary tracks, replay sampling, and the full baseline roster.

Benchmark settings. The multi-head GLUE-8 benchmark (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, QNLI, RTE, excluding WNLI as is common) (Wang et al., 2018) uses RoBERTa-base as the encoder backbone (Liu et al., 2019): the shared encoder is merged and evaluated with each task's own trained head. This multi-head setup is convenient and common, but it assumes oracle task identity at evaluation time and should not be the only evidence for the method. The recent merging literature evaluates on three benchmark families—CLIP/ViT vision task-vector suites, GLUE and text-to-text NLP, and decoder-only LLM merging, the last standardized by MergeBench with released full-parameter Llama/Gemma experts (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024; Wang et al., 2019; Yu et al., 2023; He et al., 2025)—and we position the proposal against all three.

Secondary benchmark settings. Two secondary tracks address the limitations of multi-head GLUE. A CLIP/ViT vision track evaluates the standard task-vector image-classification suite—SUN397, Cars, RESISC45, EuroSAT, SVHN, GTSRB, MNIST, DTD—testing a different modality, different loss geometry, and stronger domain shifts (Radford et al., 2021; Ilharco et al., 2022; Tang et al., 2024), together with its twenty-task extension (Wang et al., 2024) (adding CIFAR-10/100, STL10, Flowers102, Oxford Pets, PCAM, FER2013, EMNIST, Food101, FashionMNIST, KMNIST, RenderedSST2), which probes the regime in which merging degrades most: with twenty task vectors competing for the same encoder weights, cross-task conflicts dominate and each vector must be down-weighted heavily to coexist (results in Section 5.1). A decoder-only LLM track adopts the MergeBench setting (He et al., 2025), whose published full-parameter domain experts share a common pretrained checkpoint, so task vectors and merge-to-expert directions are well defined and no expert training is required on our side. We work at a full-fine-tuning-feasible scale (Llama-3.2-3B) with the mathematics, coding, instruction-following, and multilingual experts, merging the full model including the LM head; evaluation uses judge-free automatic metrics and subsampled evaluation sets, since generation-based scoring is the dominant cost. This track removes the oracle-task-identity assumption of multi-head GLUE at a scale and modality the vision suites do not reach.

Replay data. For refinement, we use $n \in \{8, 16\}$ replay examples per task in the main grids (the twenty-task budget study extends to $n = 32$; Appendix E), sampled from training data or from a held-out split separated from the final validation split, and class-balanced whenever possible. Sensitivity to the replay-budget size and to the buffer draw is reported for each track, rather than only the best tuned replay size.

Baselines. Because the proposed method uses labeled replay buffers, baselines are grouped by data regime: *checkpoint-only* (pretrained encoder, single-task experts, uniform averaging, task arithmetic, TIES, DARE, DELLA-style sampling, Model Breadcrumbs, magnitude-based Localize-and-Stitch, DOGE/APGD); *data-dependent but label-free* (RegMean, AdaMerging, APL); *labeled* (ordinary replay gradient descent from the same initialization, distance-scaled descent without the gate, head-only calibration, encoder and encoder-plus-head replay fine-tuning, multitask replay fine-tuning from the pretrained checkpoint and from the merge, Fisher merging with an empirical diagonal Fisher estimated from per-example labeled-loss gradients on the replay buffer, and Localize-and-Stitch with masks learned on the replay buffer); and *sanity controls* (inverted gate, density-matched random gate, wrong-expert directions, shuffled labels). All labeled-replay baselines use the same replay examples, the same number of gradient evaluations where possible, and the same hyperparameter search budget.

For DOGE/APGD we port the authors' released `DOGE_TA` implementation: two-dimensional encoder weights only; per-layer $\lambda_i^l = \eta / \|\tau_i^l\|$; task and shared SVD ranks $\min(d_l)/T$ and $\min(d_l)/6$; 400 Adam iterations at learning rate 10^{-4} with gradients projected orthogonally to the shared basis; and the released inclusive global top-30% task-vector mask applied to each adjusted vector. We sweep $\eta \in \{0.075, 0.10, 0.15, 0.20\}$ on GLUE-8, $\{0.03, 0.05, 0.07, 0.09, 0.11, 0.13, 0.15, 0.18\}$ on CLIP-8, and $\{0.0025, 0.005, 0.0075, 0.01, 0.015, 0.02, 0.025, 0.03, 0.05\}$ on CLIP-20. The last grid is an audited extension after the initial $\{0.03, \ldots, 0.18\}$ grid hit its lower boundary; the boundary run was stopped before evaluation, and only the winner of the expanded grid was evaluated.

D Per-Task Results at the 8+8 Budget

Tables 5–7 give the task-level scores underlying the GLUE-8, CLIP-8, and MergeBench columns of Table 2. Each row is the cell selected using the disjoint eight-example selection buffer; the evaluation scores below did not participate in selection. For GLUE-8, "score" denotes the benchmark metric used in the aggregate: Matthews correlation for CoLA, correlation for STS-B, and accuracy or accuracy/F1 for the remaining tasks. CLIP-8 reports top-1 accuracy, and MergeBench reports the judge-free automatic task scores described in Section 4.

E Twenty-Task CLIP: Extended Results

This appendix gives the full twenty-task campaign behind Section 5.1.

Table 5: GLUE-8 at the split-selected 8+8 budget: raw per-task evaluation scores for every nonempty GLUE-8 row of Table 2. The single-task-expert row evaluates each task's own expert and is included as a ceiling reference.

Initialization	Treatment	CoLA	SST-2	MRPC	STS-B	QQP	MNLI	QNLI	RTE
Pretrained	alone	0.000	0.509	0.748	−0.047	0.316	0.336	0.495	0.473
Single-task expert	alone	0.623	0.936	0.916	0.909	0.901	0.873	0.924	0.798
Pretrained	+GD	0.000	0.509	0.158	0.107	0.473	0.355	0.495	0.527
	+ungated	0.000	0.592	0.695	0.169	0.566	0.343	0.489	0.531
	+APR	0.033	0.787	0.692	0.741	0.572	0.397	0.532	0.458
Task arithmetic	alone	0.130	0.845	0.719	0.529	0.371	0.698	0.648	0.477
	+GD	0.057	0.888	0.755	0.715	0.812	0.715	0.793	0.531
	+ungated	0.017	0.878	0.775	0.821	0.792	0.747	0.738	0.513
	+APR	0.079	0.869	0.718	0.763	0.812	0.742	0.732	0.534
Breadcrumbs	alone	0.142	0.894	0.388	0.423	0.336	0.665	0.496	0.509
	+GD	−0.003	0.883	0.759	0.777	0.794	0.710	0.735	0.545
	+ungated	0.031	0.883	0.775	0.808	0.788	0.696	0.743	0.556
	+APR	0.065	0.867	0.739	0.810	0.803	0.717	0.763	0.563
DARE-TIES	alone	0.268	0.886	0.690	0.681	0.366	0.571	0.591	0.451
	+GD	0.194	0.896	0.789	0.822	0.639	0.611	0.714	0.448
	+ungated	0.129	0.893	0.762	0.809	0.780	0.655	0.772	0.523
	+APR	0.149	0.881	0.794	0.825	0.789	0.679	0.785	0.534
TIES	alone	0.195	0.862	0.712	0.286	0.330	0.461	0.570	0.458
	+GD	0.146	0.870	0.762	0.802	0.715	0.534	0.725	0.440
	+ungated	0.170	0.860	0.740	0.768	0.715	0.555	0.761	0.451
	+APR	0.190	0.860	0.767	0.795	0.748	0.581	0.745	0.477
AdaMerging	alone	−0.018	0.532	0.243	0.770	0.307	0.774	0.882	0.686
	+GD	−0.049	0.577	0.253	0.682	0.312	0.758	0.874	0.661
	+ungated	−0.026	0.588	0.731	0.815	0.750	0.756	0.883	0.588
	+APR	0.010	0.751	0.758	0.789	0.773	0.797	0.835	0.603

Table 6: CLIP-8 at the split-selected 8+8 budget: per-task top-1 accuracy for every nonempty CLIP-8 row of Table 2. The single-task-expert row evaluates each task's own expert and is included as a ceiling reference.

Initialization	Treatment	SUN397	Cars	RESISC	EuroSAT	SVHN	GTSRB	MNIST	DTD
Zero-shot	alone	0.632	0.597	0.582	0.450	0.315	0.325	0.482	0.439
Single-task expert	alone	0.749	0.783	0.949	0.991	0.972	0.989	0.996	0.794
Zero-shot	+GD	0.575	0.403	0.527	0.626	0.431	0.326	0.557	0.385
	+ungated	0.601	0.521	0.556	0.643	0.471	0.356	0.565	0.400
	+APR	0.617	0.512	0.640	0.818	0.776	0.662	0.928	0.447
Task arithmetic	alone	0.570	0.553	0.628	0.734	0.778	0.685	0.961	0.472
	+GD	0.553	0.391	0.692	0.868	0.856	0.691	0.753	0.487
	+ungated	0.555	0.450	0.659	0.847	0.830	0.748	0.947	0.477
	+APR	0.618	0.598	0.716	0.856	0.880	0.781	0.959	0.517
Breadcrumbs	alone	0.641	0.613	0.694	0.791	0.769	0.668	0.949	0.521
	+GD	0.637	0.605	0.734	0.862	0.836	0.749	0.946	0.522
	+ungated	0.617	0.521	0.735	0.860	0.826	0.747	0.940	0.502
	+APR	0.619	0.511	0.652	0.850	0.863	0.685	0.930	0.477
DARE-TIES	alone	0.594	0.577	0.663	0.713	0.862	0.792	0.977	0.486
	+GD	0.063	0.011	0.275	0.496	0.239	0.092	0.291	0.232
	+ungated	0.392	0.018	0.301	0.517	0.738	0.472	0.841	0.303
	+APR	0.646	0.538	0.773	0.867	0.902	0.860	0.971	0.522
TIES	alone	0.667	0.636	0.732	0.738	0.859	0.800	0.969	0.528
	+GD	0.666	0.630	0.779	0.876	0.898	0.848	0.960	0.539
	+ungated	0.655	0.305	0.762	0.828	0.877	0.820	0.950	0.514
	+APR	0.676	0.581	0.787	0.866	0.891	0.844	0.962	0.549
AdaMerging	alone	0.627	0.652	0.661	0.849	0.834	0.760	0.977	0.469
	+GD	0.624	0.645	0.710	0.896	0.877	0.813	0.965	0.481
	+ungated	0.620	0.334	0.716	0.904	0.866	0.804	0.965	0.469
	+APR	0.649	0.655	0.731	0.899	0.875	0.814	0.972	0.504

The vision track scales to the twenty-task suite of Wang et al. (2024) using the published fine-tuned ViT-B/32 encoders for all twenty tasks (every expert was validated to beat its zero-shot floor by a wide margin, including the three tasks whose class-name orderings we had to reconstruct: PCAM $0.556 \to 0.887$, FER2013 $0.376 \to 0.711$, RenderedSST2 $0.586 \to 0.713$).

Two protocol corrections forced by the scale. First, the uniform $\lambda_i = 0.3$ that is standard at eight tasks is catastrophic at twenty: the task-arithmetic merge scores 0.364 mean top-1 accuracy, *below* the 0.550 zero-shot floor, and the tuned optimum falls to $\lambda \approx 0.05$–0.08 (accuracy 0.604; the curve is monotone over $0.3 \to 0.08$ and flat below). Merge coefficients must be re-tuned per task count, and refinement must never start from a hand-picked global λ. Second, normalized retention (Eq. (6)) becomes *degenerate* at this scale: the suite contains tasks whose expert barely improves on the zero-shot backbone (STL10 gap 0.0043, Food101 gap 0.035), so the denominator explodes—STL10 alone produces per-task retentions near -60 and is the "worst task" of nearly every method for that reason only. On this suite we therefore report *absolute* mean and worst-task top-1 accuracy (zero-shot floor 0.550, expert ceiling 0.896) as primary, with retention restricted to non-degenerate tasks as a secondary check.

Why RegMean is an awkward anchor. RegMean is the initialization at which the anchored update is least advantaged, and the reason is specific to how RegMean sits in parameter space. RegMean solves each linear layer by matching *outputs* on that layer's inputs, $W = (\sum_i G_i)^{-1} \sum_i G_i W_i$; in input directions with negligible activation variance the solution is essentially unconstrained, so W acquires large components that are functionally inert on in-distribution inputs. The resulting merge point is over 450 from the base model in parameter norm—larger than the encoder norm itself (336)—while scoring a healthy 0.723, which is only possible because most of that displacement lies in inference-invisible directions. Every ingredient of APR reads the anchor $v_i = \theta_i - \theta$, so from this initialization v_i inherits that inflated, non-functional component: the step $-g \odot |v|$ scales noise on directions where $|v|$ is huge but the gradient is near zero, and the trust region $|v|$ is widest exactly there—which is also why the usable rate collapses by an order of magnitude here. This yields a concrete prediction: anchoring on the functional part of v (equivalently, refining RegMean from a task-vector-defined anchor) should remove the handicap—an experiment we leave to future work. It also implies that parameter-space distance from base is a valid drift proxy for the task-vector-based methods but not for RegMean, whose functional drift must be measured in output space.

Where small buffers start to overfit. A methodological point matters here: best-per-cell tables cannot detect overfitting, because selected configurations exclude it by construction, so we audit *every* (η, S, n) cell for the overfitting signature (replay loss improving from $S = 30$ to $S = 60$ while test accuracy falls). For unanchored GD the signature appears as soon as the buffer is small: 9 of 36 small-n cells deteriorate, with textbook cases at $n = 16$ (replay $0.083 \to 0.063$ while test falls $0.636 \to 0.288$; a second cell falls $0.614 \to 0.540$), plus two outright instabilities where replay worsens as well. A practical corollary is that GD's safe learning-rate window narrows as sweeps accumulate—a rate tuned at $S = 5$ cannot be reused at $S = 60$. For the anchored update the signature never appears: across ≈ 48 cells the largest decline is 0.003, with no instabilities, *even though* APR also reaches near-interpolation (replay 0.007–0.011 on $n = 8$ buffers) and its optimal rate transfers unchanged across horizons. The absence of overfitting is therefore a property of the trust region, not of the data regime: once every coordinate is capped at $|v_{i,r}|$, additional sweeps have nowhere damaging to go, whereas GD keeps moving after it has fit the buffer.

Geometry of the twenty-task regime. Direct measurement explains why the anchor's advantage concentrates on refinement from the pretrained model. At the tuned merge, every expert sits at distance ≈ 2.4 (encoder norm 336), but the best refined solutions found by *any* stable method lie only 0.14–0.23 from a strong initialization—about 8% of one expert distance. As tasks accumulate, the achievable joint optimum stays close to a good initialization (a compromise among twenty mutually incompatible experts, mirrored by the tuned λ falling from 0.2–0.3 to 0.08), while the expert distances $|v_i|$ that set APR's stride stay large; the two length scales that coincided at three and eight tasks decouple at twenty. This is the mechanism behind the shrinking gate advantage and the shrinking margin over GD from a merge, and it makes a testable prediction: from the *pretrained model*, where the destination genuinely is far, the distance-scaled step should dominate small-step GD again. It

does—the margin is $\approx +0.05$–0.06 mean accuracy at every budget (Table 8), because the good region is genuinely far (> 1) while GD's small steps travel only 0.09–0.22 and cannot reach it.

Replay budget at twenty tasks. We run a full budget grid: $n \in \{8, 16, 32\}$ labeled examples per task, each refined at $S \in \{5, 30, 60\}$ sweeps with the learning rate tuned per cell, from all four starting points, with identical buffer draws across runs (single seed). Table 8 reports the best configuration over the learning rate *and* the horizon, since the two interact: at small n nearly every best cell uses $S \geq 30$—more passes genuinely compensate for fewer examples—and the optimal rate falls as the buffer shrinks (to $\eta \approx 1$–4), while extending further to $S = 60$ moves best configurations by at most 0.004 at any budget, so the horizon is effectively converged by thirty sweeps. Three regularities. First, degradation with budget is graceful in the mean but concentrated in the *worst task*: from AdaMerging, mean accuracy loses 0.015 across the $4\times$ budget reduction ($0.698 \rightarrow 0.683$) while worst-task accuracy loses more than twice as much in relative terms ($0.241 \rightarrow 0.207$)—the cost of a small buffer is paid disproportionately by the hardest task, which a twenty-task mean conceals. Second, the refinement's value survives at the smallest budget: with just *eight* labeled examples per task, refining the base model reaches 0.632 (0.626 ± 0.005 over five buffer draws, above the merge on every draw)—while the tuned task-arithmetic merge (0.604) uses no labels but requires all twenty expert checkpoints and a tuned coefficient. Third, from the base model the anchored update's margin over plain gradient descent is large and essentially independent of the budget ($\approx +0.05$–0.06 mean at every n; GD row in Table 8): the geometry argument above, not data volume, sets that gap. Over five pinned buffer draws at $n = 8$ that margin is $+0.053 \pm 0.006$ (APR 0.626 ± 0.005 against GD 0.572 ± 0.003), and APR's worst-task accuracy is higher on every draw (0.170 ± 0.017 against 0.095 ± 0.011).

Summary of the twenty-task study. Four claims survive the campaign, each with its evidence. (i) *The anchored refinement improves every initialization it is given*, including the pretrained model itself, with the gain shrinking as the initialization improves but never vanishing (Table 8, and the CLIP-20 column of Table 1). (ii) *The load-bearing ingredient is the anchored, distance-scaled, clipped step, not the attribution gate*: the gate is neutral on mean accuracy at this scale, and its thresholded variants cannot help, because the gradient's coordinate-level selection carries no transferable structure (the held-out audit above). (iii) *Safety is the method's clearest practical advantage*: across every learning rate, horizon, and budget tested, no anchored-refinement run fell below its initialization or the zero-shot floor, and none showed the overfitting signature, while unconstrained descent deteriorates on a quarter of its small-buffer cells. (iv) *The method works in the small-data regime*: eight labeled examples per task suffice to beat the tuned task-arithmetic merge from the pretrained model alone, by $+0.053 \pm 0.006$ over ordinary descent across five buffer draws, with the budget's cost concentrated in worst-task accuracy.

Limitations. Four gaps remain. The budget grid of Table 8 is single-seed, with five-seed replication only at its from-pretrained $n = 8$ cell, and the cells of Table 1 are single-draw throughout, as are the held-out retention study of Table 4 and the drift probe of Table 3. On the twenty-task suite, the coefficient grid offered to split selection for task arithmetic ends at $\lambda = 0.2$, above that benchmark's optimum, so the selected merge is a boundary pick that lands below the zero-shot floor, and the from-task-arithmetic refinement rows of Table 1 inherit that starting point; a selection grid extended below $\lambda = 0.2$ would need those cells re-run. The predicted functional-anchor fix for the RegMean initialization is untested. On MergeBench, the five checkpoint-only/label-free baselines and the APR composition rows are included, but the ungated and ordinary-GD composition arms are absent; the decoder track therefore supports composition but not the gate-versus-anchor decomposition away from the pretrained initialization. Its composition gains are also small in the mean everywhere (at most $+0.005$) and concentrated in the worst task; task arithmetic, AdaMerging, and Breadcrumbs are mean-neutral (0.000, -0.001, and -0.005). The held-out retention study is single-seed, at one budget, on one split, and that split is a comparatively low-interference regime—eight training tasks, where merging has less to repair and the strongest label-free merge is correspondingly hard to beat; whether the ordering against AdaMerging changes as interference grows is untested here. RegMean's retention is likewise budget-dependent: worst in the table at an eight-input Gram budget, it behaves quite differently at larger ones. A further limitation is one of scope rather than execution: the results reported here are confined to budgets of at most 32 labeled examples per task, so we make no claim

about how the ordering between the anchored and unconstrained families behaves when labeled data is plentiful.

Table 7: MergeBench at the split-selected $8{+}8$ budget: per-task judge-free evaluation scores for every nonempty MergeBench row of Table 2. The single-task-expert row evaluates the corresponding expert in each column. Mean is the unweighted mean used in Table 2.

Initialization	Treatment	Math	Coding	Instruction	Multilingual	Mean
Pretrained	alone	0.063	0.387	0.210	0.495	0.289
Single-task expert	alone	0.730	0.480	0.490	0.494	0.548
Pretrained	+APR	0.310	0.393	0.283	0.503	0.373
Task arithmetic	alone	0.613	0.430	0.333	0.504	0.470
	+APR	0.560	0.463	0.350	0.505	0.470
Breadcrumbs	alone	0.467	0.447	0.253	0.502	0.417
	+APR	0.433	0.407	0.303	0.504	0.412
DARE-TIES	alone	0.677	0.440	0.350	0.493	0.490
	+APR	0.673	0.440	0.367	0.500	0.495
TIES	alone	0.583	0.423	0.357	0.492	0.464
	+APR	0.543	0.453	0.377	0.497	0.468
AdaMerging	alone	0.680	0.457	0.357	0.491	0.496
	+APR	0.633	0.477	0.373	0.495	0.495

Table 8: Replay-budget study on the twenty-task suite: best configuration over learning rate and horizon $S \in \{5, 30, 60\}$ per cell, mean top-1 accuracy (worst-task in parentheses); single seed, identical buffer draws across cells at each n. "Init. alone" is budget-independent. Ordinary GD is reported from the base model, the setting in which the anchored step's advantage is largest. Worst-task accuracy degrades faster than the mean as the buffer shrinks.

Initialization	Method	n=8	n=16	n=32
Base model (0.550 (0.100))	+ GD	0.573 (0.093)	0.580 (0.113)	0.602 (0.160)
	+ APR	**0.632** (0.187)	**0.640** (0.205)	**0.654** (0.236)
Task arith. (0.604 (0.115))	+ APR	0.650 (0.197)	0.659 (0.242)	0.674 (0.274)
TIES (0.626 (0.150))	+ APR	0.665 (0.224)	0.673 (0.250)	0.691 (0.284)
AdaMerging (0.665 (0.101))	+ APR	0.683 (0.207)	0.688 (0.249)	0.698 (0.241)